LivAna

Backend Implementation Plan

Version 1.0 · 3 Developers · 3 Phases

Spring Boot 3.x · PostgreSQL 16 · Java 21 · Valkey 7 · Oracle Cloud

3 Developers	14 Tables	58 APIs	₹0 / month
Dev1 · Dev2 · Dev3	PostgreSQL schema	Across 4 modules	Oracle Free Tier

HOW TO USE THIS DOCUMENT

This plan is written for three developers building LivAna 1.0 from scratch. All three complete Phase 1 together. From Phase 2 onwards, each developer works on their own module in parallel. Follow the steps in order — do not skip testing checkpoints.

Navy header	Dev 1 work — Auth, Security & User Management
Green header	Dev 2 work — Business, Listings & Media
Amber header	Dev 3 work — Orders, Bookings, Reviews & Notifications
■ Checkpoint	Testing checkpoint — do not proceed until all checks pass
⚠ Warning	Common mistake — read carefully before continuing
✓ Tip	Helpful note or clarification
$ command	Run this exact command in your terminal

⚠ All three developers MUST complete Phase 1 together before splitting. Skipping setup causes hours of debugging later. Sit together, one person drives, the others watch and catch mistakes.

✓ Whenever stuck for more than 30 minutes on the same error — stop and discuss with the team. Do not debug alone for hours.

PHASE 1 · ALL THREE DEVELOPERS · Project Setup & Development Environment

Sit together for this phase. One person drives, the others watch and catch mistakes. This phase takes 3–4 hours. Do it once. After this, you split.

1 Install Required Software (All Machines)

Every developer installs all of the following on their own laptop before anything else:

- IntelliJ IDEA Community (free) from jetbrains.com/idea — the IDE for Spring Boot development
- OpenJDK 21 from adoptium.net — choose the LTS release, Temurin distribution
- Maven 3.9+ from maven.apache.org — build tool for the Spring Boot project
- Docker Desktop from docker.com — runs PostgreSQL 16 and Valkey locally without installing them
- DBeaver Community from dbeaver.io — visual PostgreSQL browser, essential for inspecting data
- Postman from postman.com — how you will test every API endpoint
- Git from git-scm.com

```
$ java --version       # Must show: openjdk 21.x.x
$ mvn --version        # Must show: Apache Maven 3.9.x
$ git --version        # Must show: git version 2.x.x
$ docker --version     # Must show: Docker version 24.x.x or higher
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	java --version shows openjdk 21.x.x — not Java 8, 11, or 17
■	mvn --version returns without errors
■	Docker Desktop is running (whale icon visible in taskbar/menu bar)
■	DBeaver opens without errors

2 Create the GitHub Repository

- One person creates a new GitHub repository named livana-backend — set it to Private
- Settings → Collaborators → Add all other developers' GitHub usernames
- All developers clone the repo to their machine
- Create a .gitignore at the root (IntelliJ will generate one — accept it)

```
$ git clone https://github.com/YOUR_ORG/livana-backend.git
$ cd livana-backend
$ ls    # Should show only .gitignore initially
```

3 Create the Project Structure

One person generates the Spring Boot project using Spring Initializr (start.spring.io) with these exact settings:

- Project: Maven Language: Java Spring Boot: 3.3.x
- Group: com.livana Artifact: backend Java: 21
- Dependencies to add: Spring Web, Spring Data JPA, Spring Security, PostgreSQL Driver, Validation, Lombok, Spring Boot Actuator

```
# After unzipping the generated project into the repo root:
livana-backend/
├── src/main/java/com/livana/
│   ├── auth/          ← Dev 1 owns this package
│   ├── user/          ← Dev 1 owns this package
│   ├── business/      ← Dev 2 owns this package
│   ├── listing/       ← Dev 2 owns this package
│   ├── order/         ← Dev 3 owns this package
│   ├── booking/       ← Dev 3 owns this package
│   ├── review/        ← Dev 3 owns this package
│   ├── notification/  ← Dev 3 owns this package
│   ├── common/        ← shared utilities (all devs read, no one deletes)
│   └── config/        ← Spring config classes (Dev 1 owns)
├── src/main/resources/
│   ├── application.properties
│   └── application-dev.properties
└── db/
    └── schema.sql     ← copy the finalised schema here
```

✓ Each developer owns their packages. Never edit another developer's package without telling them. The common/ package is read-only for Dev 2 and Dev 3 — only Dev 1 adds to it after discussion.

4 Set Up Docker Compose for Local Development

Create a file called docker-compose.yml in the ROOT of the project. This starts PostgreSQL 16 and Valkey on your local machine with one command. You do NOT need to install PostgreSQL or Valkey directly — Docker handles everything.

```
# docker-compose.yml — place in project root
version: '3.9'
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: livana
      POSTGRES_USER: livana_user
      POSTGRES_PASSWORD: livana_local_pass
    ports:
      - '5432:5432'
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./db:/docker-entrypoint-initdb.d   # runs schema.sql on first start

  valkey:
    image: valkey/valkey:7
    ports:
      - '6379:6379'
```

```
volumes:
  pgdata:
```

```
$ docker compose up -d        # Start both in background
$ docker compose ps           # Both should show 'running'
$ docker compose logs postgres  # Check for errors
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	docker compose ps shows both postgres and valkey as 'running'
■	Open DBeaver → New Connection → PostgreSQL → Host: localhost, Port: 5432, DB: livana, User: livana_user, Password: livana_local_pass → Test Connection → Success
■	redis-cli -p 6379 ping → returns PONG (Valkey is Redis-compatible)

5 Load the Database Schema

- Copy the finalised schema.sql into the /db folder
- Rename it to 01_schema.sql — Docker runs files in /db alphabetically on first start
- Add seed data for cities and amenities as 02_seed.sql

```
$ cp schema.sql db/01_schema.sql
$ docker compose down
$ docker volume rm livana-backend_pgdata    # wipe old empty DB
$ docker compose up -d                      # restarts + runs schema
```

```
# 02_seed.sql — create this file in /db
INSERT INTO cities (name, state) VALUES
  ('Dehradun', 'Uttarakhand'),
  ('Roorkee',  'Uttarakhand'),
  ('Haridwar', 'Uttarakhand');

INSERT INTO amenities (name, icon_name) VALUES
  ('WiFi','wifi'),('AC','ac'),('Geyser','geyser'),
  ('Laundry','laundry'),('Parking','parking'),('CCTV','cctv'),
  ('Meals Included','meals'),('Power Backup','power'),('Gym','gym');
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	DBeaver → expand connection → Schemas → public → Tables → all 14 tables visible
■	Right-click the cities table → View Data → Dehradun, Roorkee, Haridwar rows exist
■	Right-click the amenities table → View Data → 9 amenity rows exist
■	If tables are missing: run docker compose logs postgres and fix the SQL error shown

6 Configure application.properties

Create two property files — one for local development, one that will be used in production. Never commit secrets to GitHub.

```
# src/main/resources/application.properties (shared, committed to GitHub)
spring.application.name=livana-backend
spring.profiles.active=${SPRING_PROFILE:dev}
server.port=8080
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
management.endpoints.web.exposure.include=health,info

# src/main/resources/application-dev.properties (local only — do NOT commit)
spring.datasource.url=jdbc:postgresql://localhost:5432/livana
spring.datasource.username=livana_user
spring.datasource.password=livana_local_pass
spring.data.redis.host=localhost
spring.data.redis.port=6379
app.jwt.secret=LOCAL_DEV_SECRET_CHANGE_IN_PROD
app.jwt.access-token-expiry-ms=900000
app.jwt.refresh-token-expiry-ms=604800000
app.google.client-id=YOUR_GOOGLE_CLIENT_ID_HERE
```

⚠ Add application-dev.properties to .gitignore immediately. It contains your local credentials. Never commit this file. Production credentials will be set as environment variables on the Oracle VM, not in any file.

7 Verify Spring Boot Starts

Before splitting, verify that the base Spring Boot application starts without errors and the health endpoint responds.

```
$ mvn spring-boot:run
# You should see:  Started LivanaApplication in X.XXX seconds

# In a new terminal:
$ curl http://localhost:8080/actuator/health
# Expected response: {"status":"UP"}
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	mvn spring-boot:run starts without BUILD FAILURE
■	curl http://localhost:8080/actuator/health returns {"status":"UP"}
■	DBeaver still shows all 14 tables (Spring validate mode didn't break anything)
■	No red errors in IntelliJ — only possible warnings about unused config are fine

8 Establish Git Workflow

- Never commit directly to the main branch — this breaks the other developers' work

- Dev 1 works on branch: feature/dev1 — creates it now
- Dev 2 works on branch: feature/dev2 — creates it now
- Dev 3 works on branch: feature/dev3 — creates it now
- When a feature is complete: create a Pull Request to main → team reviews → merge
- Every morning before starting: git pull origin main and merge into your branch

```
# Dev 1 runs:
$ git checkout -b feature/dev1 && git push origin feature/dev1

# Dev 2 runs:
$ git checkout -b feature/dev2 && git push origin feature/dev2

# Dev 3 runs:
$ git checkout -b feature/dev3 && git push origin feature/dev3
```

✓	Agree on commit message format now: feat: add listing search API, fix: null pointer in order service, test: add booking endpoint test, chore: update dependencies. This makes the git history readable.

✓	TESTING CHECKPOINT — do not proceed until all pass
■	All three developers can see each other's branches on GitHub
■	Test merge: Dev 1 adds one line to README.md, pushes to feature/dev1, creates PR to main, Dev 2 or Dev 3 approves and merges
■	Dev 2 and Dev 3 pull from main and see the README change — confirms the workflow works
■	Phase 1 is complete. You now have a running local DB, Valkey, and Spring Boot. Split from here.

PHASE 2 · DEV 1 · Auth, Security & User Management

You own the security foundation of LivAna. Every other developer's API depends on your JWT filter being correct. Build auth first, then security config, then user APIs, then rate limiting. Do not skip testing at each step — a broken JWT filter breaks all 58 endpoints.

1 Add Required Dependencies to pom.xml

Open pom.xml and add these dependencies inside the <dependencies> block:

```
<dependency>
  <groupId>com.google.auth</groupId>
  <artifactId>google-auth-library-oauth2-http</artifactId>
  <version>1.23.0</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.12.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.12.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-core</artifactId>
  <version>8.10.1</version>
</dependency>
<dependency>
  <groupId>com.bucket4j</groupId>
  <artifactId>bucket4j-redis</artifactId>
  <version>8.10.1</version>
</dependency>
```

```
$ mvn dependency:resolve    # Should complete without BUILD FAILURE
```

2 Build the JWT Service

Create com.livana.auth.JwtService — the core class that signs and validates tokens. This is the most important class Dev 1 writes. All protected endpoints depend on it.

```
@Service
public class JwtService {

    @Value("${app.jwt.secret}")
    private String secret;

    @Value("${app.jwt.access-token-expiry-ms}")
    private long accessTokenExpiry;     // 15 minutes = 900000ms

    public String generateAccessToken(Long userId, String role) {
        return Jwts.builder()
            .subject(userId.toString())
            .claim("role", role)
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + accessTokenExpiry))
            .signWith(getKey())
            .compact();
    }

    public Claims validateToken(String token) {
        return Jwts.parser()
            .verifyWith(getKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getKey() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }
}
```

⚠ The JWT secret must be at least 32 characters long — JJWT will throw a WeakKeyException if it is shorter. In production on the Oracle VM, set this as an environment variable APP_JWT_SECRET with a randomly generated 64-character string.

3 Build Google Token Validator

Create com.livana.auth.GoogleTokenValidator — validates the Google ID token the app sends after the user taps 'Sign in with Google'. This never calls Google's API directly — it validates the token cryptographically using Google's public keys.

```
@Component
public class GoogleTokenValidator {

    @Value("${app.google.client-id}")
    private String clientId;

    public GoogleIdToken.Payload validate(String idTokenString) {
        try {
```

```
            GoogleIdTokenVerifier verifier = new GoogleIdTokenVerifier.Builder(
                    new NetHttpTransport(), new GsonFactory())
                .setAudience(Collections.singletonList(clientId))
                .build();
            GoogleIdToken token = verifier.verify(idTokenString);
            if (token == null) throw new UnauthorizedException("Invalid Google token");
            return token.getPayload();
        } catch (Exception e) {
            throw new UnauthorizedException("Google token validation failed");
        }
    }
}
```

✓ To get a Google Client ID: go to console.cloud.google.com → APIs & Services → Credentials → Create Credentials → OAuth 2.0 Client ID → Android/iOS app type. Add the client ID to application-dev.properties.

4 Build the Refresh Token Service (Valkey)

Refresh tokens are long-lived (7 days) and stored in Valkey — not in the database. This allows instant revocation: to log a user out, just delete their Valkey key.

```
@Service
@RequiredArgsConstructor
public class RefreshTokenService {

    private final StringRedisTemplate redisTemplate;
    private static final Duration TTL = Duration.ofDays(7);

    public String create(Long userId) {
        String token = UUID.randomUUID().toString();
        redisTemplate.opsForValue()
            .set("refresh:" + token, userId.toString(), TTL);
        return token;
    }

    public Long validateAndGetUserId(String token) {
        String val = redisTemplate.opsForValue().get("refresh:" + token);
        if (val == null) throw new UnauthorizedException("Refresh token invalid or
expired");
        return Long.parseLong(val);
    }

    public void revoke(String token) {
        redisTemplate.delete("refresh:" + token);
    }
}
```

5 Build Auth Endpoints (POST /api/auth/google, /refresh, /logout)

Create com.livana.auth.AuthController with three endpoints. These are the only endpoints that do NOT require a JWT — they are permit-all in Spring Security.

```
POST /api/auth/google
  Request body:  { "idToken": "<Google ID token from app>" }
  Logic:
    1. Call GoogleTokenValidator.validate(idToken)
    2. Extract email and google_id from the payload
    3. Upsert user: INSERT INTO users ... ON CONFLICT (google_id) DO UPDATE SET ...
    4. Generate JWT access token (15 min) via JwtService
    5. Generate refresh token (7 days) via RefreshTokenService, store in Valkey
    6. Return { accessToken, refreshToken, user: { id, name, email, role } }

POST /api/auth/refresh
  Request body:  { "refreshToken": "<UUID>" }
  Logic:
    1. Call RefreshTokenService.validateAndGetUserId(refreshToken)
    2. Load user from DB by ID
    3. Generate new access token
    4. Return { accessToken }

POST /api/auth/logout
  Requires: valid JWT in Authorization header
  Request body:  { "refreshToken": "<UUID>" }
  Logic:
    1. Call RefreshTokenService.revoke(refreshToken)
    2. Return 200 OK
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	POST /api/auth/google with a real Google ID token from your test Android/iOS app → accessToken and refreshToken returned
■	Open DBeaver → users table → new row visible with your google_id and email
■	POST /api/auth/refresh with the returned refreshToken → new accessToken returned
■	POST /api/auth/logout → 200 OK; then POST /api/auth/refresh with same token → 401 Unauthorized
■	Calling a protected endpoint (e.g. GET /api/users/me) without Authorization header → 401

6 Configure Spring Security Filter Chain

Create com.livana.config.SecurityConfig — this is the central security configuration that Dev 2 and Dev 3 depend on. Get this right before they start coding.

```
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers(GET, "/api/listings/**").permitAll()
                .requestMatchers(GET, "/api/businesses/**").permitAll()
                .requestMatchers(GET, "/api/cities").permitAll()
                .requestMatchers(GET, "/api/amenities").permitAll()
                .requestMatchers(GET, "/actuator/health").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

```
// JwtAuthFilter — reads the Authorization header and sets the security context
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(HttpServletRequest req,
            HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            try {
                Claims claims = jwtService.validateToken(token);
                Long userId = Long.parseLong(claims.getSubject());
                String role = claims.get("role", String.class);
                var auth = new UsernamePasswordAuthenticationToken(
                    userId, null,
                    List.of(new SimpleGrantedAuthority("ROLE_" + role)));
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                res.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Invalid token");
                return;
            }
        }
        chain.doFilter(req, res);
    }
}
```

⚠ Share the SecurityConfig with Dev 2 and Dev 3 immediately after it is committed. They need it to test their endpoints. Tell them: any endpoint that should be public needs a permitAll() line here. They ask you, you add it.

7 Build User Profile APIs

```
GET  /api/users/me      → return currently logged-in user's profile
PUT  /api/users/me      → update full_name, phone, avatar_url, city_id
DELETE /api/users/me    → set is_active = false (soft delete)
```

```
// Helper — extract userId from security context in any controller:
Long userId = (Long) SecurityContextHolder.getContext()
                        .getAuthentication().getPrincipal();
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	GET /api/users/me with valid JWT → returns your user profile
■	PUT /api/users/me with { "phone": "9876543210" } → phone updated in DB (verify in DBeaver)
■	GET /api/users/me with expired JWT → 401 Unauthorized
■	GET /api/users/me with no header → 401 Unauthorized

8 Implement Bucket4j Rate Limiting

Rate limiting prevents a buggy app client or malicious user from overwhelming your Oracle VM. Implement this before Dev 2 and Dev 3 finish their endpoints so it is in place from the start.

```
@Component
@RequiredArgsConstructor
public class RateLimitInterceptor implements HandlerInterceptor {

    private final ProxyManager<String> proxyManager;

    @Override
    public boolean preHandle(HttpServletRequest req,
            HttpServletResponse res, Object handler) {

        String key = resolveKey(req);
        Bucket bucket = proxyManager.builder()
            .build(key, () -> BucketConfiguration.builder()
                .addLimit(Bandwidth.simple(200, Duration.ofMinutes(1)))
                .build());

        if (bucket.tryConsume(1)) return true;

        res.setStatus(429);
        res.setHeader("Retry-After", "60");
        return false;
    }

    private String resolveKey(HttpServletRequest req) {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth != null && auth.isAuthenticated() &&
            auth.getPrincipal() instanceof Long userId) {
            return "rl:user:" + userId;
        }
        return "rl:ip:" + req.getRemoteAddr();
    }
}
```

```
# Rate limit configuration (application.properties):
```

```
# General authenticated:     200 requests / 1 minute  per user
# Search/browse listings:     60 requests / 1 minute  per user
# Order placement:            10 requests / 1 minute  per user
# Authentication attempts:    10 requests / 15 minutes per IP
# Image upload:               20 requests / 1 hour    per user
# Unauthenticated endpoints:  30 requests / 1 minute  per IP
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	Send 201 rapid requests to any endpoint → 201st returns HTTP 429 with Retry-After: 60 header
■	Wait 1 minute → requests succeed again
■	Verify in Valkey: redis-cli keys 'rl:*' shows your rate limit keys

9 Build Cities API and Admin User Endpoints

```
GET  /api/cities                        → list all cities (public, no auth)

# Admin endpoints — require ROLE_ADMIN
GET  /api/admin/users                   → list all users (cursor paginated)
PUT  /api/admin/users/{id}/ban          → set is_active = false
PUT  /api/admin/users/{id}/unban        → set is_active = true
PUT  /api/admin/users/{id}/role         → body: { "role": "BUSINESS_OWNER" }
```

✓	To test admin endpoints locally: manually update your user's role to ADMIN in DBeaver (UPDATE users SET role = 'ADMIN' WHERE id = 1), then log in again to get a new JWT with the ADMIN role claim.

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	GET /api/cities → returns list of seeded cities without authentication
■	GET /api/admin/users with STUDENT JWT → 403 Forbidden
■	GET /api/admin/users with ADMIN JWT → list of users returned
■	PUT /api/admin/users/{id}/ban → user's is_active becomes false in DBeaver
■	Banned user tries to call GET /api/users/me → 401 Unauthorized (check is_active on every auth)

PHASE 2 · DEV 2 · Business, Listings & Media

You own everything a business owner does — registration, managing their profile, adding listings, setting hours, and uploading photos. You also own the search and browse experience for students. Start with business CRUD, then listings, then search, then media upload last.

⚠ Before writing any code, pull Dev 1's SecurityConfig and JwtAuthFilter from main. You need them to test your protected endpoints. Run: git checkout main && git pull && git checkout feature/dev2 && git merge main

1 Build the Standard Response Wrapper

Create com.livana.common.ApiResponse — a shared response wrapper used by all controllers. Commit it to main immediately so Dev 1 and Dev 3 can also use it.

```
@Getter
@AllArgsConstructor
public class ApiResponse<T> {
    private final boolean success;
    private final String message;
    private final T data;

    public static <T> ApiResponse<T> ok(T data) {
        return new ApiResponse<>(true, "Success", data);
    }
    public static <T> ApiResponse<T> created(T data) {
        return new ApiResponse<>(true, "Created", data);
    }
    public static ApiResponse<Void> ok(String message) {
        return new ApiResponse<>(true, message, null);
    }
}

// Usage in any controller:
return ResponseEntity.ok(ApiResponse.ok(businessDto));
return ResponseEntity.status(201).body(ApiResponse.created(listingDto));
```

2 Build Business Registration and CRUD APIs

Create the Business entity, repository, service, and controller. The 14-table schema is already in PostgreSQL — you only write Java code that maps to it.

```
POST /api/businesses
  Auth: BUSINESS_OWNER or STUDENT (becomes BUSINESS_OWNER after first business)
  Body: { name, type, description, address, city_id, latitude, longitude, phone }
  Logic:
    1. Validate all required fields with @Valid + @NotBlank annotations
    2. Set owner_id = currently logged-in userId
    3. Set is_verified = false (requires admin approval)
```

```
    4. Save business
    5. Auto-create a default business_settings row with all notify flags = true
    6. Auto-create 7 business_hours rows (Mon–Sun), all is_closed = false, 09:00–22:00
    7. Return 201 Created with the new business

GET  /api/businesses/{id}    → public, no auth needed
PUT  /api/businesses/{id}    → owner only (check owner_id == logged-in userId)
PUT  /api/businesses/{id}/toggle   → flip is_open boolean, owner only
```

⚠	Always verify ownership before allowing PUT or DELETE. Pattern: if (!business.getOwnerId().equals(loggedInUserId)) throw new ForbiddenException("Not your business"). Never trust the client to send the correct owner ID.

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	POST /api/businesses with valid body and BUSINESS_OWNER JWT → 201, business created in DBeaver
■	DBeaver → business_settings table → one row auto-created for the new business
■	DBeaver → business_hours table → 7 rows auto-created (Mon–Sun) for the new business
■	GET /api/businesses/{id} without auth → 200 OK (public endpoint)
■	PUT /api/businesses/{id} with a different user's JWT → 403 Forbidden

3 Build Business Hours APIs

```
GET /api/businesses/{id}/hours
  → returns all 7 day rows for this business, no auth required

PUT /api/businesses/{id}/hours
  Auth: owner only
  Body: array of 7 day objects:
  [
    { "day": "MONDAY",    "is_closed": false, "open_time": "09:00", "close_time": "22:00"
},
    { "day": "SUNDAY",    "is_closed": true,  "open_time": null,    "close_time": null
},
    ...5 more days
  ]
  Logic: run 7 individual UPDATEs or use a batch update
  Validate: if is_closed = true, open_time and close_time must be null
```

4 Build Listings CRUD APIs

One listing table serves all three business types (PG, MESS, GROCERY). The listing_metadata JSONB column holds type-specific fields. Validate the metadata structure based on the parent business type.

```
POST /api/businesses/{businessId}/listings
  Auth: owner of that business only
  Body: { title, description, price, category, listing_metadata, images[] }
  Validate listing_metadata structure based on business.type:
```

```
    PG      → requires room_type, gender_preference, security_deposit
    MESS    → requires plan_duration, meal_types, is_veg_only
    GROCERY → requires unit, stock_quantity

GET  /api/businesses/{businessId}/listings  → all listings for a business (public)
PUT  /api/listings/{id}                     → owner only
DELETE /api/listings/{id}                   → sets is_active = false, owner only
PUT  /api/listings/{id}/toggle              → flips is_available, owner only
```

5 Build Search & Browse API with Cursor-Based Pagination

This is the most-called endpoint in the entire system. Every time a student opens the app, this runs. Get it right and get it fast.

```
GET /api/listings
  Query params: city_id, type, min_price, max_price, amenities (comma-separated IDs),
                category, cursor (last seen listing id), limit (default 20, max 50)

  SQL — cursor-based pagination (NOT OFFSET):
  SELECT l.*, b.name AS business_name, b.rating, b.is_open
  FROM listings l
  JOIN businesses b ON l.business_id = b.id
  WHERE b.city_id = :cityId
    AND b.type    = :type
    AND l.is_active = true
    AND l.is_available = true
    AND l.price BETWEEN :minPrice AND :maxPrice
    AND l.id > :cursor       ← cursor pagination (replace OFFSET with this)
  ORDER BY l.id ASC
  LIMIT :limit;

  Response: { items: [...], nextCursor: 47, hasMore: true }
  nextCursor = the id of the last item in this page
  Client sends ?cursor=47 on the next request to get the next page
```

⚠	Never use OFFSET-based pagination (LIMIT x OFFSET y). At page 50 with 1000 rows offset, PostgreSQL reads and discards 1000 rows on every request. Cursor pagination always reads only the rows it returns. The schema already has an index on listings(id ASC, is_active) for this query.

✓	TESTING CHECKPOINT — do not proceed until all pass
■	GET /api/listings?city_id=1&type=MESS → returns listings, each with business name and rating
■	GET /api/listings?city_id=1&type=PG&min_price=5000&max_price=15000 → filtered results
■	Get first page, take nextCursor, call GET /api/listings?cursor={value} → different results (next page)
■	GET /api/listings?city_id=1&type=PG&amenities=1,2 → only PG listings with both amenities
■	Run the query in DBeaver with EXPLAIN ANALYZE — confirm it uses the index, not a seq scan

6 Implement Valkey Caching for Listings

Add cache-aside caching to the search/browse endpoints. This reduces DB queries by 70–80% for popular searches.

```
// In ListingService.searchListings():

String cacheKey = buildCacheKey(cityId, type, cursor, filters);
// e.g. "listings:1:MESS:0:500-2000"

// 1. Check cache first
String cached = redisTemplate.opsForValue().get(cacheKey);
if (cached != null) {
    return objectMapper.readValue(cached, ListingPageDto.class);
}

// 2. Cache miss — query DB
ListingPageDto result = listingRepository.search(params);

// 3. Store in cache with TTL
redisTemplate.opsForValue().set(cacheKey, objectMapper.writeValueAsString(result),
    Duration.ofMinutes(5));

return result;

// Cache invalidation — call this after any listing update:
// redisTemplate.delete("listings:" + cityId + ":" + type + ":*")
// Use RedisTemplate.keys() to find and delete matching keys
```

7 Build Amenities API and Listing Amenities

```
GET /api/amenities                        → public, returns master amenities list
PUT /api/listings/{id}/amenities          → owner only
  Body: { "amenity_ids": [1, 2, 4, 7] }
  Logic: DELETE all existing listing_amenities for this listing,
         then bulk INSERT the new set
  Only applicable for PG type listings — validate parent business type
```

8 Build Media Upload Flow (Cloudflare R2)

Images are NEVER uploaded through your Spring Boot backend. The backend only issues a pre-signed URL. The client uploads directly to Cloudflare R2. This keeps your Oracle VM bandwidth near zero.

```
# Add to pom.xml:
<dependency>
  <groupId>software.amazon.awssdk</groupId>
  <artifactId>s3</artifactId>  <!-- R2 is S3-compatible -->
  <version>2.25.60</version>
</dependency>

# Add to application.properties:
r2.account-id=YOUR_CLOUDFLARE_ACCOUNT_ID
r2.access-key=YOUR_R2_ACCESS_KEY
r2.secret-key=YOUR_R2_SECRET_KEY
r2.bucket=livana-media
r2.public-url=https://pub-XXXX.r2.dev
```

```
POST /api/media/upload-url
  Auth: any authenticated user
  Body: { "file_type": "image/jpeg", "file_size_bytes": 524288 }
  Logic:
    1. Reject if file_size_bytes > 2MB (2097152 bytes)
    2. Generate a unique object key: media/{userId}/{UUID}.jpg
    3. Create pre-signed S3 PUT URL (valid 5 minutes) using AWS SDK v2
    4. Return { upload_url, object_key, expires_in: 300 }

POST /api/media/confirm
  Auth: any authenticated user
  Body: { "object_key": "media/42/abc123.jpg" }
  Logic:
    1. Build public R2 URL: {r2.public-url}/{object_key}
    2. Return { public_url }
  Note: caller saves this URL to the relevant DB record themselves
        (listing.images[], businesses.cover_url, users.avatar_url)
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	POST /api/media/upload-url with valid JWT → upload_url returned (starts with your R2 bucket URL)
■	Use the returned upload_url in Postman → PUT request with a real JPEG file → 200 from Cloudflare
■	POST /api/media/confirm with the object_key → public_url returned
■	Visit the public_url in a browser → the image loads directly from Cloudflare (not your backend)
■	POST /api/media/upload-url with file_size_bytes > 2MB → 400 Bad Request

9 Build Business Settings and Admin Verification APIs

```
GET  /api/businesses/{id}/settings    → owner only
PUT  /api/businesses/{id}/settings    → owner only
  Body: { notify_new_order, notify_new_review, notify_booking_request,
          razorpay_account_id, payment_onboarded }

# Admin endpoints:
GET  /api/admin/businesses            → paginated list, ADMIN only
PUT  /api/admin/businesses/{id}/verify   → set is_verified = true
PUT  /api/admin/businesses/{id}/unverify → set is_verified = false
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	Full end-to-end test: Register a business → Upload a cover photo → Add listings → Set hours → Toggle is_open
■	Student flow: GET /api/listings → see the new business's listings in search results
■	Verify Valkey caching: same search twice → second call is measurably faster; check redis-cli keys 'listings:*'
■	Admin verifies business: GET /api/businesses/{id} → is_verified now true

PHASE 2 · DEV 3 · Orders, Bookings, Reviews & Notifications

You own the transactional core of LivAna — what happens after a student finds what they want. Orders and bookings have strict status machines. Reviews update aggregate data. Notifications fire on every key event. Build in this order: orders → bookings → reviews → notifications.

⚠ Pull from main before starting. You need Dev 1's JWT filter and Dev 2's ApiResponse. Run: git checkout main && git pull && git checkout feature/dev3 && git merge main

1 Build Order Placement API

Orders are for MESS and GROCERY only. A student picks items, places an order, and the business owner fulfills it. The entire order (header + items) must be created in a single database transaction.

```
POST /api/orders
  Auth: STUDENT
  Body:
  {
    "business_id": 5,
    "delivery_address": "Room 12, Block B",
    "note": "No onion please",
    "items": [
      { "listing_id": 23, "quantity": 2 },
      { "listing_id": 24, "quantity": 1 }
    ]
  }

  Logic (ALL inside a single @Transactional method):
    1. Load all listings by ID, verify they belong to the same business_id
    2. Verify all listings have is_active = true AND is_available = true
    3. Calculate total_amount = sum of (listing.price * quantity)
    4. INSERT into orders with status = PENDING
    5. For each item: INSERT into order_items with
       title_snapshot = listing.title, price_snapshot = listing.price
    6. Fire notification to business owner (ORDER_PLACED)
    7. Return 201 with the created order
```

⚠ Always use @Transactional on the service method that creates orders. If the notification fails or any item insert fails, the entire order must roll back. A partial order with no items is a data corruption bug.

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	POST /api/orders with valid body → 201; check DBeaver: orders table has new row, order_items has matching rows
■	POST /api/orders with a listing from a different business_id → 400 Bad Request
■	POST /api/orders with an unavailable listing → 400 Bad Request

■	GET /api/orders/{id} → order detail with items array returned
■	GET /api/orders/my → student's order history returned

2 Build Order Status Machine

Only the business owner can advance order status. A student can only cancel (if still PENDING). Status must move forward — you cannot go from DELIVERED back to PENDING.

```
Status flow:
  PENDING → CONFIRMED → PREPARING → READY → DELIVERED
       ↘                                  ↗
         CANCELLED  (from PENDING or CONFIRMED only)

PUT /api/orders/{id}/status   (BUSINESS_OWNER only)
  Body: { "status": "CONFIRMED" }
  Validation: allowed transitions only
    PENDING    → CONFIRMED, CANCELLED
    CONFIRMED  → PREPARING, CANCELLED
    PREPARING  → READY
    READY      → DELIVERED
  Fire notification to student on every status change

PUT /api/orders/{id}/cancel   (STUDENT only)
  Allowed only when current status = PENDING
  Set cancelled_by = student's userId, prompt for cancel_reason
  Fire notification to business owner (ORDER_CANCELLED)
```

```
// Status transition validation — put in OrderService:
private static final Map<OrderStatus, Set<OrderStatus>> ALLOWED_TRANSITIONS = Map.of(
    PENDING,   Set.of(CONFIRMED, CANCELLED),
    CONFIRMED, Set.of(PREPARING, CANCELLED),
    PREPARING, Set.of(READY),
    READY,     Set.of(DELIVERED)
);

public void validateTransition(OrderStatus from, OrderStatus to) {
    if (!ALLOWED_TRANSITIONS.getOrDefault(from, Set.of()).contains(to)) {
        throw new BadRequestException(
            "Cannot transition from " + from + " to " + to);
    }
}
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	PENDING → CONFIRMED by business owner → order status updated in DBeaver
■	CONFIRMED → DELIVERING (invalid) → 400 Bad Request with clear error message
■	Student cancels PENDING order → CANCELLED, cancelled_by set to student's id
■	Student tries to cancel CONFIRMED order → 403 Forbidden
■	Business owner status update fires FCM notification to student (verify in FCM console logs)

3 Build Business Order Management APIs

```
GET /api/businesses/{businessId}/orders
  Auth: owner of that business only
  Query params: status (filter), cursor, limit (default 20)
  Returns: paginated list of orders with student name and item count
  Note: use cursor-based pagination same as Dev 2's listing search

GET /api/orders/{id}
  Auth: accessible by the student who placed it OR the business owner
  Returns: full order with order_items, each item showing title_snapshot and
price_snapshot
```

4 Build PG Booking APIs

Bookings are for PG accommodation only. The lifecycle is different from orders — a booking lasts months, not minutes. There is no items table for bookings — one row covers the entire stay.

```
POST /api/bookings
  Auth: STUDENT
  Body:
  {
    "listing_id": 12,
    "check_in_date": "2026-06-01",
    "duration_months": 6,
    "note": "I will arrive after 5 PM"
  }
  Logic:
    1. Load listing, verify it belongs to a PG type business
    2. Verify listing is_active = true and is_available = true
    3. Verify student has no other ACTIVE or CONFIRMED booking (one at a time rule)
    4. Set monthly_rent_snapshot = listing.price at time of booking
    5. Calculate check_out_date = check_in_date + duration_months
    6. INSERT booking with status = PENDING
    7. Fire notification to PG owner (BOOKING_REQUEST)
    8. Return 201

PUT /api/bookings/{id}/confirm     → BUSINESS_OWNER only (PENDING → CONFIRMED)
PUT /api/bookings/{id}/status      → BUSINESS_OWNER only (CONFIRMED → ACTIVE →
COMPLETED)
PUT /api/bookings/{id}/cancel      → STUDENT or BUSINESS_OWNER
GET /api/bookings/my               → student's booking history (cursor paginated)
GET /api/businesses/{id}/bookings  → owner's incoming bookings (cursor paginated)
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	POST /api/bookings → 201; DBeaver shows booking with PENDING status and correct monthly_rent_snapshot
■	Post second booking while first is PENDING → 400 (student can have only one active booking at a time)
■	Owner confirms booking → CONFIRMED; student receives FCM notification
■	GET /api/bookings/my → student's booking history returned with correct check_out_date
■	Try to book a GROCERY listing → 400 Bad Request (bookings only for PG)

5 Build Reviews API

A student can leave one review per business. Submitting a review must atomically update the business's rating and total_reviews in the same transaction.

```
POST /api/reviews
  Auth: STUDENT
  Body:
  {
    "business_id": 5,
    "rating": 4,
    "comment": "Good food, delivery was a bit slow",
    "category_scores": [
      { "category": "Food",    "score": 5 },
      { "category": "Service", "score": 3 },
      { "category": "Hygiene", "score": 4 }
    ]
  }

  Logic (inside @Transactional):
    1. Verify student does not already have a review for this business
       (DB has UNIQUE constraint — but check first for a clean error message)
    2. INSERT into reviews
    3. INSERT into review_category_scores (one row per category)
    4. Recalculate business aggregate:
       UPDATE businesses
       SET rating = (SELECT ROUND(AVG(rating)::numeric, 1) FROM reviews WHERE business_id
= :id),
           total_reviews = total_reviews + 1
       WHERE id = :businessId
    5. Fire notification to business owner (REVIEW_RECEIVED)
    6. Invalidate Valkey cache key for this business's rating

GET  /api/businesses/{id}/reviews         → public, cursor paginated
GET  /api/businesses/{id}/reviews/summary → public, returns avg rating + per-category avg
DELETE /api/reviews/{id}                  → student deletes own review (recalculate
aggregate)
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	POST /api/reviews → review created; DBeaver shows review and category_scores rows
■	DBeaver businesses table → rating and total_reviews updated correctly (not 0, not null)
■	POST second review for same business → 409 Conflict (one review per business)
■	GET /api/businesses/{id}/reviews/summary → shows per-category averages
■	DELETE review → rating recalculated downward in businesses table

6 Build Notifications API

Notifications are stored in PostgreSQL for in-app history AND sent via FCM for real-time push. Always do both — the stored notification is the history feed, FCM is the ping.

```
# Add FCM dependency to pom.xml:
<dependency>
  <groupId>com.google.firebase</groupId>
  <artifactId>firebase-admin</artifactId>
  <version>9.3.0</version>
</dependency>

# Add Firebase service account JSON path to application.properties:
firebase.service-account-path=classpath:firebase-service-account.json

# Download the service account JSON from:
# Firebase Console → Project Settings → Service accounts → Generate new private key
# Place in src/main/resources/ and add to .gitignore
```

```
@Service
@RequiredArgsConstructor
public class NotificationService {

    private final NotificationRepository notifRepo;
    private final UserRepository userRepo;

    public void send(Long userId, NotificationType type, Object payloadObj) {
        // 1. Store in DB
        Notification notif = new Notification();
        notif.setUserId(userId);
        notif.setType(type);
        notif.setPayload(objectMapper.valueToTree(payloadObj));
        notif.setIsRead(false);
        notifRepo.save(notif);

        // 2. Send FCM push (async — do not block the main thread)
        String fcmToken = userRepo.findFcmToken(userId);
        if (fcmToken != null) {
            sendFcmAsync(fcmToken, type, payloadObj);
        }
    }

    @Async
    void sendFcmAsync(String token, NotificationType type, Object payload) {
        Message msg = Message.builder()
            .setToken(token)
            .putData("type", type.name())
            .putData("payload", objectMapper.writeValueAsString(payload))
            .build();
        FirebaseMessaging.getInstance().sendAsync(msg);
    }
}
```

✓ Add an fcm_token column to the users table (VARCHAR 255, nullable). The mobile app sends its FCM token to PUT /api/users/me when it starts. Dev 1 adds the column — coordinate this with Dev 1 early.

```
GET /api/notifications
  Auth: any user
```

```
  Query params: cursor, limit (default 20)
  Returns: cursor-paginated notification history for the logged-in user

GET  /api/notifications/unread-count   → returns { count: 7 }
PUT  /api/notifications/{id}/read      → mark one as read
PUT  /api/notifications/read-all       → mark all as read for this user
DELETE /api/notifications/{id}         → delete own notification
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	Place a test order → DBeaver notifications table has new ORDER_PLACED row for the business owner
■	FCM push arrives on the test device (check Firebase Console → Cloud Messaging for delivery stats)
■	GET /api/notifications → returns notification list ordered by created_at DESC
■	GET /api/notifications/unread-count → matches unread rows in DBeaver
■	PUT /api/notifications/read-all → all notifications for this user now is_read = true
■	Full end-to-end: Student places order → owner gets ORDER_PLACED push → owner confirms → student gets ORDER_CONFIRMED push

PHASE 3 · ALL THREE DEVELOPERS · Integration, Deployment & Go-Live

All three developers come back together here. Every module is built and tested in isolation. Now connect them, test the full flow end-to-end, and deploy to Oracle Cloud. Do not rush this phase — a stable deployment is better than a fast broken one.

1 Merge All Branches to Main

```
# Each developer creates a PR to main:
# Dev 1: feature/dev1 → main
# Dev 2: feature/dev2 → main
# Dev 3: feature/dev3 → main

# Merge order matters — merge Dev 1 first:
# 1. Merge Dev 1's PR (auth/security foundation)
# 2. Merge Dev 2's PR (resolve any conflicts with Dev 1)
# 3. Merge Dev 3's PR (resolve any conflicts)

$ git checkout main && git pull
$ mvn clean install   # Must show BUILD SUCCESS with all tests passing
```

⚠ If mvn clean install fails after merging, DO NOT proceed to deployment. Fix all compilation errors and test failures first. A failing build on main means the codebase is broken for everyone.

2 Full End-to-End Integration Test (All Together)

Run these full user journeys using Postman before touching the Oracle VM. One person runs the tests, others watch and note any failures.

```
JOURNEY 1 — Student finds and orders food from a mess:
  1. POST /api/auth/google → get JWT
  2. GET /api/cities → confirm Dehradun is listed
  3. GET /api/listings?city_id=1&type=MESS → see listings
  4. GET /api/listings/{id} → view single listing detail
  5. POST /api/orders → place order
  6. GET /api/orders/my → order appears in history
  7. [As business owner] PUT /api/orders/{id}/status → CONFIRMED
  8. [As student] GET /api/notifications → ORDER_CONFIRMED notification present

JOURNEY 2 — Student books a PG room:
  1. GET /api/listings?city_id=1&type=PG → see PG listings
  2. GET /api/listings/{id} → verify amenities in response
  3. POST /api/bookings → booking created with PENDING status
  4. [As PG owner] PUT /api/bookings/{id}/confirm → CONFIRMED
  5. [As student] GET /api/bookings/my → shows CONFIRMED booking

JOURNEY 3 — Student leaves a review:
  1. POST /api/reviews with category scores
  2. GET /api/businesses/{id} → rating updated
```

```
  3. GET /api/businesses/{id}/reviews/summary → category breakdown correct

JOURNEY 4 — Business owner uploads a photo:
  1. POST /api/media/upload-url → get pre-signed URL
  2. PUT to the pre-signed URL with a real JPEG
  3. POST /api/media/confirm → get public URL
  4. PUT /api/businesses/{id} with cover_url = public URL
  5. GET /api/businesses/{id} → cover_url visible in response
```

✓	TESTING CHECKPOINT — do not proceed until all pass
■	All 4 journeys complete without errors
■	DBeaver shows correct data after each journey
■	Notifications created for every key event
■	Valkey: run redis-cli keys '*' → see listing cache keys and rate limit keys
■	No NullPointerExceptions or 500 errors in the IntelliJ console

3 Set Up the Oracle Cloud VM

Sign up at cloud.oracle.com. Use a real email and a real credit card — you will NOT be charged for Always Free resources. The sign-up requires a credit card for identity verification only.

```
# VM Settings — choose these exactly:
  Instance shape: VM.Standard.A1.Flex (ARM Ampere)
  OCPUs: 4    RAM: 24 GB    Boot volume: 100 GB
  OS: Canonical Ubuntu 22.04 LTS (ARM)

# Download the SSH private key when shown — this is your only chance.
# Save it in a safe location. You cannot retrieve it again.

# In Networking → Security List → Add Ingress Rules for ports:
  22   (SSH)
  80   (HTTP)
  443  (HTTPS)
  8080 (Spring Boot — only temporarily, close after Nginx is set up)

# Connect to the VM:
$ chmod 400 your-key.pem
$ ssh -i your-key.pem ubuntu@YOUR_VM_PUBLIC_IP
```

```
# On the VM — install Java 21:
$ sudo apt update && sudo apt upgrade -y
$ sudo apt install openjdk-21-jdk -y
$ java --version    # Confirm: openjdk 21

# Install Docker:
$ curl -fsSL https://get.docker.com | sh
$ sudo usermod -aG docker ubuntu
$ newgrp docker
$ docker --version
```

```
# Install Nginx:
$ sudo apt install nginx -y

# Clone the repo:
$ git clone https://github.com/YOUR_ORG/livana-backend.git
$ cd livana-backend
```

✓ TESTING CHECKPOINT — do not proceed until all pass	
■	ssh into the VM succeeds
■	java --version shows 21 on the VM
■	docker --version shows 24.x.x on the VM
■	git clone succeeds and you can see the project files

4 Start PostgreSQL and Valkey on the VM

```
# On the VM, inside the livana-backend folder:
$ docker compose up -d
$ docker compose ps   # Both postgres and valkey should show 'running'

# Run the schema against the production DB:
$ docker compose exec postgres psql -U livana_user -d livana \
    -f /docker-entrypoint-initdb.d/01_schema.sql
$ docker compose exec postgres psql -U livana_user -d livana \
    -f /docker-entrypoint-initdb.d/02_seed.sql

# Add restart: always to docker-compose.yml so Docker restores
# PostgreSQL and Valkey automatically on VM reboot:
services:
  postgres:
    restart: always
    ...
  valkey:
    restart: always
    ...
```

5 Configure Production Environment and Deploy Spring Boot

```
# Create the production properties file on the VM (NEVER commit this):
$ nano livana-backend/src/main/resources/application-prod.properties

# Contents of application-prod.properties:
spring.datasource.url=jdbc:postgresql://localhost:5432/livana
spring.datasource.username=livana_user
spring.datasource.password=STRONG_PROD_PASSWORD_HERE
spring.data.redis.host=localhost
spring.data.redis.port=6379
app.jwt.secret=64_CHARACTER_RANDOM_STRING_HERE
app.jwt.access-token-expiry-ms=900000
app.jwt.refresh-token-expiry-ms=604800000
app.google.client-id=YOUR_PRODUCTION_GOOGLE_CLIENT_ID
r2.account-id=YOUR_CLOUDFLARE_ACCOUNT_ID
r2.access-key=YOUR_R2_ACCESS_KEY
```

```
r2.secret-key=YOUR_R2_SECRET_KEY
r2.bucket=livana-media
r2.public-url=https://pub-XXXX.r2.dev
firebase.service-account-path=classpath:firebase-service-account.json
spring.profiles.active=prod

# Build the JAR on the VM:
$ cd livana-backend
$ mvn clean package -DskipTests
# This creates: target/backend-0.0.1-SNAPSHOT.jar

# Run the JAR:
$ java -jar target/backend-0.0.1-SNAPSHOT.jar &
$ curl http://localhost:8080/actuator/health
# Must return: {"status":"UP"}
```

⚠ Generate the JWT secret with: openssl rand -base64 48. Never use a guessable string. If this secret is ever leaked, all user sessions are compromised and you must rotate it and force all users to log in again.

6 Configure Nginx as Reverse Proxy with SSL

```
# Install Certbot for free SSL (Let's Encrypt):
$ sudo apt install certbot python3-certbot-nginx -y

# Point your domain's DNS A record to your Oracle VM's public IP first.
# Wait 5–10 minutes for DNS to propagate, then:
$ sudo certbot --nginx -d api.yourdomain.com
# Follow prompts — Certbot auto-configures Nginx with SSL

# Create /etc/nginx/sites-available/livana:
server {
    server_name api.yourdomain.com;
    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}

$ sudo ln -s /etc/nginx/sites-available/livana /etc/nginx/sites-enabled/
$ sudo nginx -t      # Test config — must show: syntax is ok
$ sudo systemctl reload nginx

# Test SSL is working:
$ curl https://api.yourdomain.com/actuator/health
# Must return: {"status":"UP"}
```

✓ Once Nginx is working, remove port 8080 from Oracle's Security List Ingress Rules. Only ports 80 and 443 should be publicly accessible. Port 8080 is now only reachable from localhost through Nginx.

7 Set Up Automated Daily Backup to Cloudflare R2

```
# Install rclone for syncing to R2:
$ curl https://rclone.org/install.sh | sudo bash
$ rclone config   # Follow prompts to add Cloudflare R2 as 'r2' remote

# Create backup script at /home/ubuntu/backup.sh:
#!/bin/bash
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="/tmp/livana_backup_$TIMESTAMP.sql.gz"

# Dump PostgreSQL and compress
docker compose -f /home/ubuntu/livana-backend/docker-compose.yml \
  exec -T postgres pg_dump -U livana_user livana | gzip > $BACKUP_FILE

# Upload to R2
rclone copy $BACKUP_FILE r2:livana-backups/postgres/

# Clean up local file
rm $BACKUP_FILE

echo "Backup completed: $TIMESTAMP"

$ chmod +x /home/ubuntu/backup.sh

# Schedule via cron — runs at 2:00 AM daily:
$ crontab -e
# Add this line:
0 2 * * * /home/ubuntu/backup.sh >> /home/ubuntu/backup.log 2>&1
```

8 Set Up UptimeRobot Monitoring

- Sign up free at uptimerobot.com
- Add New Monitor → HTTP(s) → URL: https://api.yourdomain.com/actuator/health
- Monitoring interval: every 5 minutes
- Alert contacts: add all three developers' email addresses
- UptimeRobot sends an email if the endpoint goes down — free for up to 50 monitors

✓	Add Oracle's built-in VM monitoring too: Oracle Cloud Console → Observability → Monitoring → Create Alarm for CPU > 80% and Disk > 85%. This catches resource issues before they become outages.

9 Final Go-Live Checklist

Go through every item below. Do not go live until all are checked.

Security

■	JWT secret is a random 64-character string — not 'mysecret' or anything guessable
■	All .env and application-prod.properties are in .gitignore — never committed
■	Google OAuth client ID is correct and restricted to your app's package name
■	Rate limiting is active — test: 201 rapid requests get HTTP 429
■	Admin endpoints return 403 for STUDENT and BUSINESS_OWNER JWT tokens
■	application-dev.properties does not exist on the production VM
■	Port 8080 removed from Oracle Security List — only 80 and 443 open
■	SSL certificate installed — all traffic uses HTTPS, never HTTP

Data Integrity

■	title_snapshot and price_snapshot are correct — place a test order, change listing price, verify old order still shows original price
■	Review submission atomically updates businesses.rating — verify in DBeaver after review
■	Order creation is transactional — simulate a failure mid-order and verify no partial records
■	Booking prevents duplicate active bookings for the same student
■	Only valid order status transitions are accepted — invalid transitions return 400
■	Cursor-based pagination works — first page, then use nextCursor for second page

Operations

■	Spring Boot restarts on VM reboot: add ExecStart and Restart=always to a systemd service, or use pm2
■	Docker containers restart on VM reboot: restart: always in docker-compose.yml
■	Nginx restarts on VM reboot: sudo systemctl enable nginx
■	SSL certificate auto-renews: sudo certbot renew --dry-run returns success
■	Daily backup runs: manually run backup.sh, verify file appears in Cloudflare R2
■	UptimeRobot monitor is active and all three developers receive alert emails
■	At least one ADMIN user exists: manually set role = ADMIN for one developer's account

You built something real.

Ship the modular monolith. Get to 100 real users. Then iterate.